

Chapter 12

조인 튜닝 - 1

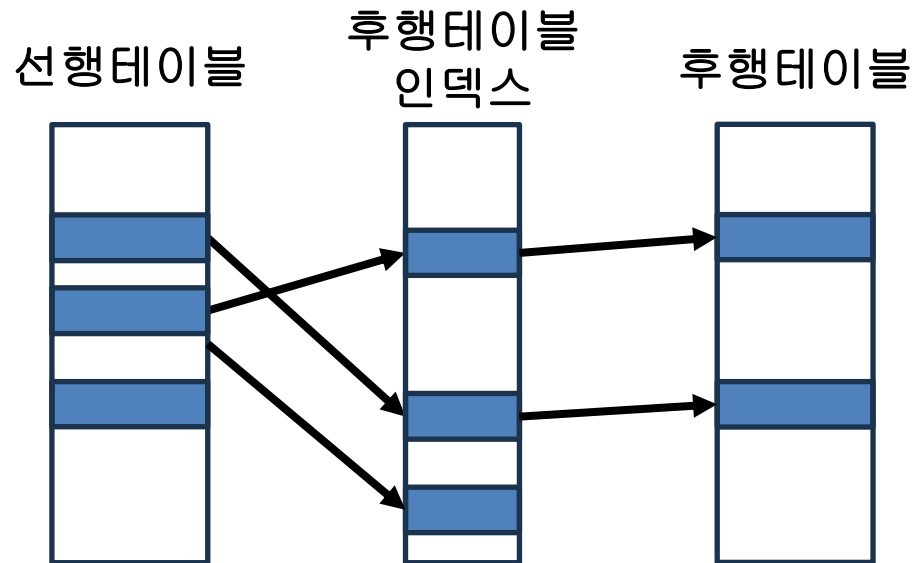
Outline

- 12.1 NL 조인
- 12.2 소트 머지 조인
- 12.3 해시 조인
- 12.4 조인 기법 선택 기준

12.1 NL 조인 (Nested Loop Join)

NL 조인 (Nested Loop Join)의 개념

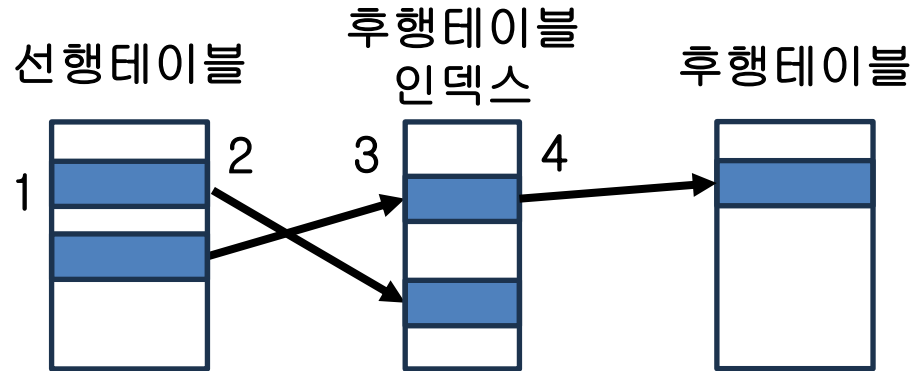
선행 테이블(Driving Table)의 결과 기준으로 후행테이블로 순차적으로 연결해 데이터를 조인하는 기법



- 프로그램의 반복문을 중첩시킨 구조와 유사한 방법으로 조인
- 선행 테이블을 얼마나 적게 조회하는지가 성능에 영향을 미침
- 후행 테이블이 인덱스가 없을 경우 심각한 성능저하가 발생

12.1 NL 조인 (Nested Loop Join)

NL 조인 (Nested Loop Join)의 조인과정



NO	설명
1	선행 테이블에서 조건을 만족하는 행을 찾음 -> 조인조건을 만족하지 않으면 필터링
2	선행 테이블의 조인 키로 후행 테이블에 조인을 시도
3	후행 테이블의 인덱스에서 선행 테이블의 조인 키가 있는지 확인 -> 조인조건을 만족하지 않으면 필터링
4	인덱스에서 추출한 레코드 정보로 후행 테이블을 액세스
5	1 ~ 4번 반복

12.1 NL 조인 (Nested Loop Join)

NL 조인 (Nested Loop Join)의 실행계획

```
explain plan set statement_id = 'sql1' for
Select *
from emp A
      , dept B
where A.deptno = B.deptno
AND A.deptno = 10
AND A.sal > 1000
ORDER BY job ASC;

select plan_table_output from table(dbms_xplan.display(null, null, 'serial'));
```

- 위 sql을 실행해 실행계획을 추출하면 어떤 과정을 통해서 NL 조인이 실행되는지 알 수 있다.

12.1 NL 조인 (Nested Loop Join)

NL 조인 (Nested Loop Join)의 실행계획 해석

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	59	4 (25)	00:00:01
1 ④	SORT ORDER BY		1	59	4 (25)	00:00:01
2	NESTED LOOPS		1	59	3 (0)	00:00:01
3 ②	TABLE ACCESS BY INDEX ROWID	DEPT	1	20	1 (0)	00:00:01
* 4 ①	INDEX UNIQUE SCAN	PK_DEPT	1		0 (0)	00:00:01
* 5 ③	TABLE ACCESS FULL	EMP	1	39	2 (0)	00:00:01

Predicate Information (identified by operation id):

4 - access("B"."DEPTNO"=10)
5 - filter("A"."DEPTNO"=10 AND "A"."SAL">1000)

- 조인 방법으로 하위의 테이블들이 Nested Loop로 조인
- ① DEPT 테이블의 PK_DEPT Object로 조회
- ② ROWID로 테이블을 Access
- ③ EMP 테이블을 FULL Scan
- ④ Nested Loop Join 후 SORT ORDER BY 실행

12.1 NL 조인 (Nested Loop Join)

NL 조인 (Nested Loop Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
Select /*+ ordered use_nl(B) */ A.EMPNO, A.DEPTNO, B.DNAME
from emp A
      , dept B
where A.deptno = B.deptno
AND A.deptno = 10;

select plan_table_output from table(dbms_xplan.display(null, null, 'serial'));
```

- USE_NL, ordered 힌트를 통해서 실행계획 제어가 가능
- **USE_NL** : NL 방식 조인 사용
- **ordered** : FROM 절에 기술된 순서대로 조인

12.1 NL 조인 (Nested Loop Join)

NL 조인 (Nested Loop Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
select /*+ leading(A, B, C) use_nl(B) use_nl(C) */
      A.EMPNO, A.DEPTNO, B.DNAME, C.EMPNO
from   emp A, dept B, emp C
where  A.deptno = B.deptno AND A.deptno = 10 AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null, null, 'serial'));
```

- 3개 이상 테이블 조인 시 **leading** 힌트를 통해 조인 순서를 제어

```
select /*+ use_nl(A, B, C) */
      A.EMPNO, A.DEPTNO, B.DNAME, C.EMPNO
from   emp A, dept B, emp C
where  A.deptno = B.deptno AND A.deptno = 10 AND A.MGR = C.EMPNO;
```

- ordered 나 leading을 사용하지 않으면 옵티마이저가 조인 순서를 결정

12.1 NL 조인 (Nested Loop Join)

NL 조인 (Nested Loop Join)의 특징

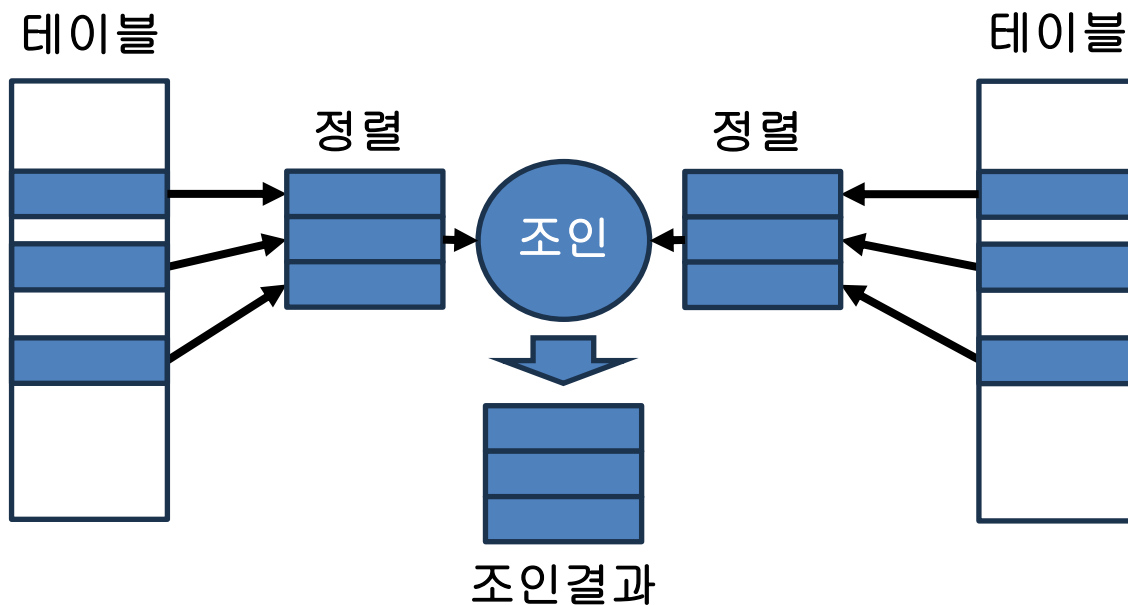
특징	설명
랜덤 액세스	- 랜덤 액세스 위주의 조인 방식이라 대량 데이터를 조인할 때 비효율 발생
순차진행	- 조인을 한 레코드 씩 순차적으로 진행 해 대용량 처리 시 성능 상 문제 가 발생 가능 - 하지만 부분범위처리가 가능 할 경우 속도 향상이 가능 - 순차진행의 특성 때문에 선행 테이블의 범위에 따라 전체 일량이 결정
인덱스 구성	- 조인 컬럼에 인덱스 여부에 따라 조인 효율이 달라짐

- 위 특징을 종합할 때 NL 조인은 소량 데이터를 조회할 때 적합한 조인기법 이다.
- **랜덤 액세스** : 한번에 하나의 블록만 액세스하는 방식
- **부분범위처리** : 조건을 만족하는 Row를 일정 사이즈만큼만 조회 (ex. ROWNUM)

12.2 소트 머지 조인 (Sort Merge Join)

소트 머지 조인 (Sort Merge Join)의 개념

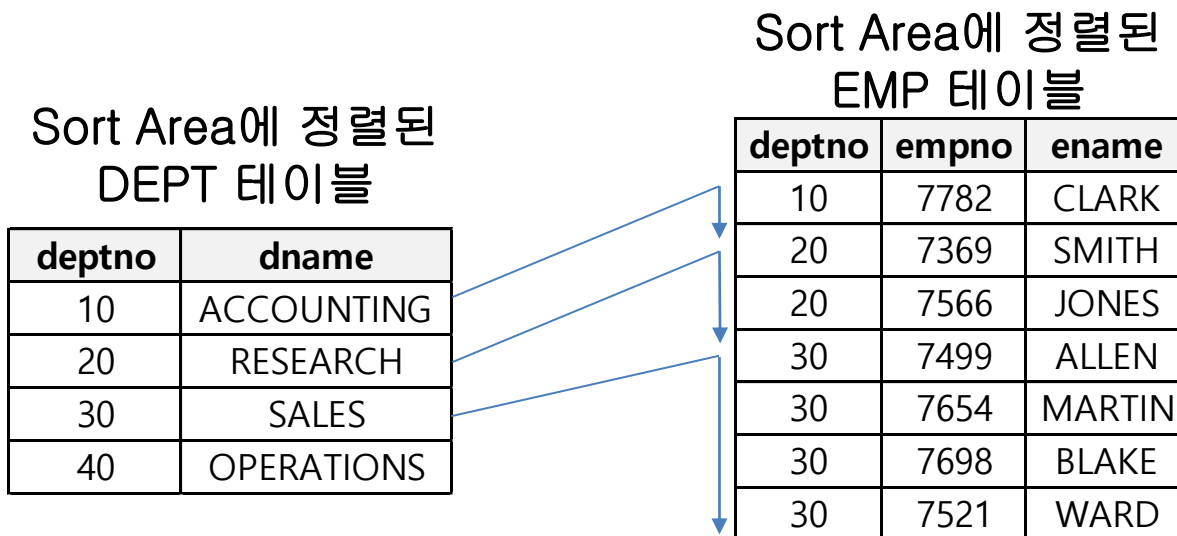
두 테이블을 먼저 각각 정렬을 한 다음에 집합을 Merge 하면서
조인하는 조인기법



- NL 조인의 랜덤 액세스와 다르게 Full Table Scan으로 조회
- 정렬할 데이터가 많아 메모리가 부족할 경우 디스크를 이용
- 비동등 조인(Non-equi join)도 조인이 가능

12.2 소트 머지 조인 (Sort Merge Join)

소트 머지 조인 (Sort Merge Join)의 조인과정



NO	설명
1	DEPT 테이블과, EMP 테이블을 조인 키로 Sort Area에서 정렬함
2	두 테이블의 조인키로 탐색을 해 조인을 함
3	이때 조인을 실패하는 레코드를 만나는 순간 멈추고 다음 레코드 탐색 시 멈춤 지점부터 다시 시작
4	반복

12.2 소트 머지 조인 (Sort Merge Join)

소트 머지 조인 (Sort Merge Join)의 실행계획

```
explain plan set statement_id = 'sql1' for
SELECT /*+ ordered use_merge(A) */
        A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME
FROM DEPT B, EMP A
WHERE A.DEPTNO = B.DEPTNO;

select plan_table_output from
table(dbms_xplan.display('plan_table',null,'serial'));
```

- 위 sql을 실행해 실행계획을 추출하면 어떤 과정을 통해서 소트 머지 조인이 실행되는지 알 수 있다.

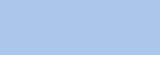
12.2 소트 머지 조인 (Sort Merge Join)

소트 머지 조인 (Sort Merge Join)의 실행계획 해석

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	182	5 (20)	00:00:01
1	MERGE JOIN		7	182	5 (20)	00:00:01
2	② TABLE ACCESS BY INDEX ROWID	DEPT	4	52	2 (0)	00:00:01
3	① INDEX FULL SCAN	PK_DEPT	4		1 (0)	00:00:01
* 4	④ SORT JOIN		7	91	3 (34)	00:00:01
5	③ TABLE ACCESS FULL	EMP	7	91	2 (0)	00:00:01

Predicate Information (identified by operation id):

4 - access("A"."DEPTNO"="B"."DEPTNO")
filter("A"."DEPTNO"="B"."DEPTNO")

-  조인 방법으로 하위의 테이블들이 MERGE JOIN
- ① DEPT 테이블의 PK_DEPT Object로 FULL SCAN 조회
- ② ROWID로 테이블을 Access
- ③ EMP 테이블을 FULL Scan
- ④ SORT JOIN 실행

12.2 소트 머지 조인 (Sort Merge Join)

소트 머지 조인 (Sort Merge Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
SELECT /*+ ordered use_merge(A) */
        A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME
FROM DEPT B, EMP A
WHERE A.DEPTNO = B.DEPTNO;

select plan_table_output from
table(dbms_xplan.display('plan_table',null,'serial'));
```

- USE_MERGE, ordered 힌트를 통해서 실행계획 제어가 가능
DEPT 테이블을 DEPTNO 컬럼으로 정렬 및
EMP 테이블을 DEPTNO 컬럼으로 정렬 후 조인
- USE_MERGE : 소트 머지 방식 조인 사용
- ordered : FROM 절에 기술된 순서대로 조인

12.2 소트 머지 조인 (Sort Merge Join)

소트 머지 조인 (Sort Merge Join)의 특징

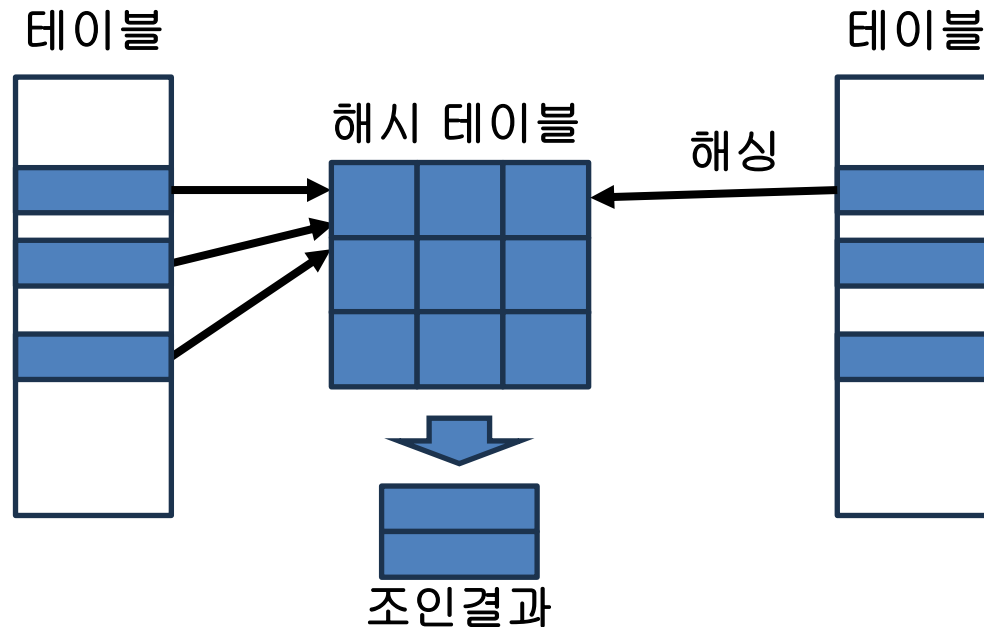
특징	설명
양쪽 정렬	- 조인하기 전에 양쪽 테이블(집합)을 조인키로 정렬을 한 후에 조인
두 테이블의 건수에 따라 일량 결정	- 양쪽 테이블(집합)을 각각 정렬한 후 조인을 하므로, 각 집합의 크기에 따라 일량 결정 - 테이블별 검색 조건에 의해 전체 일량이 달라짐
스캔방식 조인	- NL 조인은 랜덤 액세스 방식의 조인이라면 소트 머지 조인은 스캔 위주 조인 방식 - Inner 테이블을 반복 액세스 하지 않아 머지 과정에서 랜덤 액세스가 발생하지 않음 - 하지만, 각 테이블의 대상집합을 찾을 때 인덱스를 이용한 랜덤 액세스가 처리될 수 있고 이때 랜덤 액세스량이 많다면 소트 머지 조인의 이점이 사라짐

- 위 특징을 종합할 때 소트 머지 조인은 대량 데이터를 조회할 때 적합한 조인기법 이다. (배치에서 사용됨)
- 비동등 조인(Non-equi join)의 대량 데이터 조인
- 카테시안 곱
- **랜덤 액세스** : 한번에 하나의 블록만 액세스하는 방식

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 개념

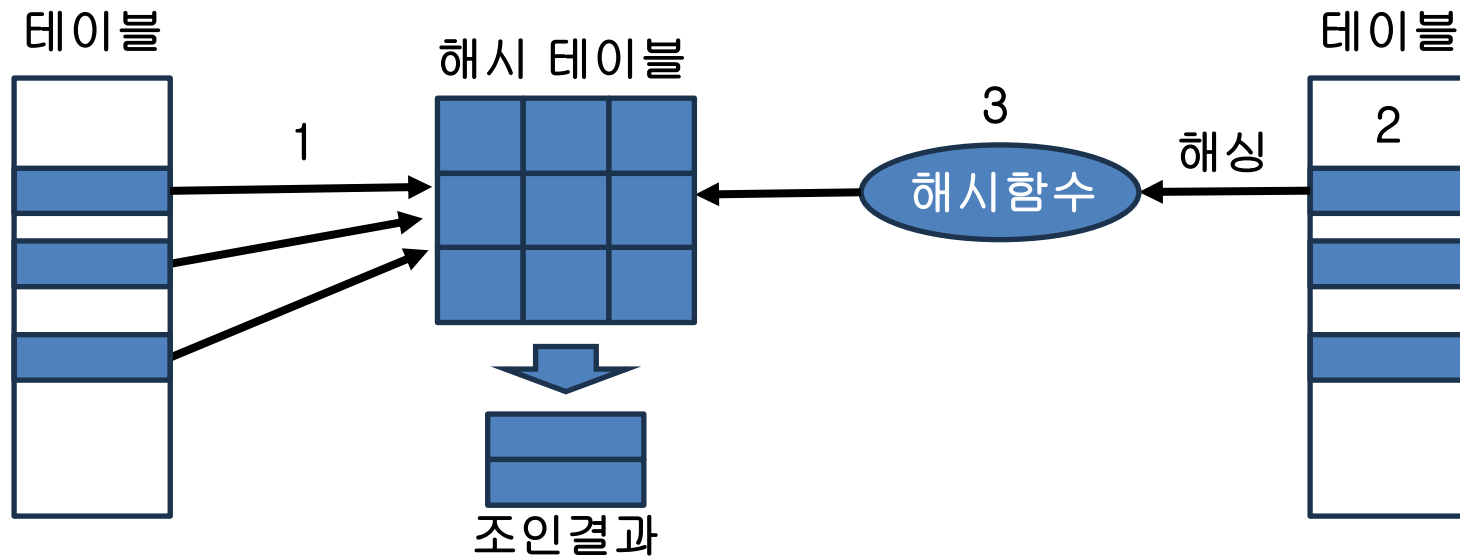
두 테이블 중 작은 테이블을 해시 영역(Hash Area)에 해시 테이블을 생성하고 큰 집합을 읽어 해시 테이블을 탐색해 조인



- NL Join의 랜덤 액세스의 문제와 소트 머지 조인의 정렬 문제점을 해결하는 대안으로 등장
- 동등 조인(Equi join)만 조인이 가능

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 조인과정



NO	설명
1	두 테이블 중 작은 테이블에서 조인 조건에 해당되는 행의 데이터로 해시 테이블을 생성(Build Input)
2	조인할 테이블에서 조인 조건에 해당하는 행을 탐색
3	조인할 테이블의 조인 컬럼 기준으로 해시함수를 적용해 해시테이블에서 적합한 버킷을 탐색
4	조인 성공 시 추출 버퍼에 저장
5	2~4번 반복

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 실행계획

```
explain plan set statement_id = 'sql1' for  
SELECT A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME  
FROM DEPT B, EMP A  
WHERE A.DEPTNO = B.DEPTNO;
```

```
select plan_table_output from  
table(dbms_xplan.display('plan_table',null,'serial'));
```

- 위 sql을 실행해 실행계획을 추출하면 어떤 과정을 통해서 해시 조인이 실행되는지 알 수 있다.

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 실행계획 해석

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	182	4 (0)	00:00:01
* 1	③ HASH JOIN		7	182	4 (0)	00:00:01
2	① TABLE ACCESS FULL	DEPT	4	52	2 (0)	00:00:01
3	② TABLE ACCESS FULL	EMP	7	91	2 (0)	00:00:01

Predicate Information (identified by operation id):

1 - access ("A"."DEPTNO"="B"."DEPTNO")

-  조인 방법으로 하위의 테이블들이 HASH JOIN
- ① DEPT 테이블 FULL Scan
- ② EMP 테이블 FULL Scan
- ③ HASH JOIN

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for  
SELECT /*+ ordered use_hash(A) */ A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME  
FROM DEPT B, EMP A  
WHERE A.DEPTNO = B.DEPTNO;  
  
select plan_table_output from table(dbms_xplan.display(null,null, 'serial'));
```

- USE_HASH, ordered 힌트를 통해서 실행계획 제어가 가능
DEPT 테이블과 EMP 테이블 중 작은 테이블을 build input으로
선택 후 해시테이블을 생성 후 조인
- **USE_HASH** : 해시 방식 조인 사용
- **ordered** : FROM 절에 기술된 순서대로 조인

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for  
SELECT /*+ ordered use_hash(A) swap_join_inputs(A) */  
        A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME  
FROM DEPT B, EMP A  
WHERE A.DEPTNO = B.DEPTNO;  
  
select plan_table_output from table(dbms_xplan.display(null,null, 'serial'));
```

– SWAP_JOIN_INPUTS : build input 테이블을 선택

```
SELECT /*+ leading(B, A, C) use_hash(A) use_hash(c) */  
        A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME  
FROM DEPT B, EMP A, EMP C  
WHERE A.DEPTNO = B.DEPTNO  
AND A.MGR = C.EMPNO;
```

- 3개 이상 테이블을 조인하면 leading 힌트로 조인순서를 지정
- Leading 힌트의 첫번째 파라미터 테이블은 build input 선택

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) swap_join_inputs(B) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM   DEPT B, EMP A, EMP C
WHERE  A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
```

– build input 테이블에 따라 실행계획이 달라짐

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	280	6 (0)	00:00:01
* 1	HASH JOIN		7	280	6 (0)	00:00:01
* 2	HASH JOIN		7	210	4 (0)	00:00:01
3	TABLE ACCESS FULL DEPT		4	52	2 (0)	00:00:01
4	TABLE ACCESS FULL EMP		7	119	2 (0)	00:00:01
5	TABLE ACCESS FULL EMP		7	70	2 (0)	00:00:01

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) swap_join_inputs(C) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM   DEPT B, EMP A, EMP C
WHERE  A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
```

– build input 테이블에 따라 실행계획이 달라짐

Id	Operation	Name	Rows	Bytes	Cost	(%CPU)	Time
0	SELECT STATEMENT		7	280	6	(0)	00:00:01
* 1	HASH JOIN		7	280	6	(0)	00:00:01
2	TABLE ACCESS FULL	EMP	7	70	2	(0)	00:00:01
* 3	HASH JOIN		7	210	4	(0)	00:00:01
4	TABLE ACCESS FULL	DEPT	4	52	2	(0)	00:00:01
5	TABLE ACCESS FULL	EMP	7	119	2	(0)	00:00:01

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 실행계획 제어

```
explain plan set statement_id = 'sql1' for
SELECT /*+ leading(B, A, C) no_swap_join_inputs(C) */
      A.DEPTNO, B.DNAME, A.EMPNO, A.ENAME, A.MGR, C.ENAME
FROM   DEPT B, EMP A, EMP C
WHERE  A.DEPTNO = B.DEPTNO AND A.MGR = C.EMPNO;

select plan_table_output from table(dbms_xplan.display(null,null,'serial'));
```

– build input 선택하지 않을 경우 no_swap_join_inputs 힌트 사용

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		7	280	6 (0)	00:00:01
* 1	HASH JOIN		7	280	6 (0)	00:00:01
* 2	HASH JOIN		7	210	4 (0)	00:00:01
3	TABLE ACCESS FULL DEPT		4	52	2 (0)	00:00:01
4	TABLE ACCESS FULL EMP		7	119	2 (0)	00:00:01
5	TABLE ACCESS FULL EMP		7	70	2 (0)	00:00:01

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)의 메모리 초과 시 처리방식

메모리 초과로 In-Memory 해시 조인이 불가능할 때 ‘Grace 해시 조인’이라는 알고리즘을 사용

처리방식	설명
파티션 단계	- 조인되는 양쪽 집합의 조인 컬럼에 해시 함수를 적용하고 해시 값에 따라 파티셔닝 - 양쪽 집합을 모두 읽어 디스크 상의 Temp 공간에 저장해서 In-Memory 해시 조인 보다 성능이 떨어짐
조인 단계	- 각 파티션 짝에 대해 하나씩 조인을 수행 - 파티션 별 작은 쪽을 Build Input으로 하여 해시 테이블을 생성 - 해시 테이블 생성이 되면 다른쪽 Probe Input으로 로우를 하나씩 읽으며 해시 테이블 조회

- 위 특징을 종합할 때 해시 조인은 대량 데이터를 조회할 때 적합한 조인기법 이다. (단, 등치 조건일 경우에만)

12.3 해시 조인 (Hash Join)

해시 조인 (Hash Join)이 빠른 이유

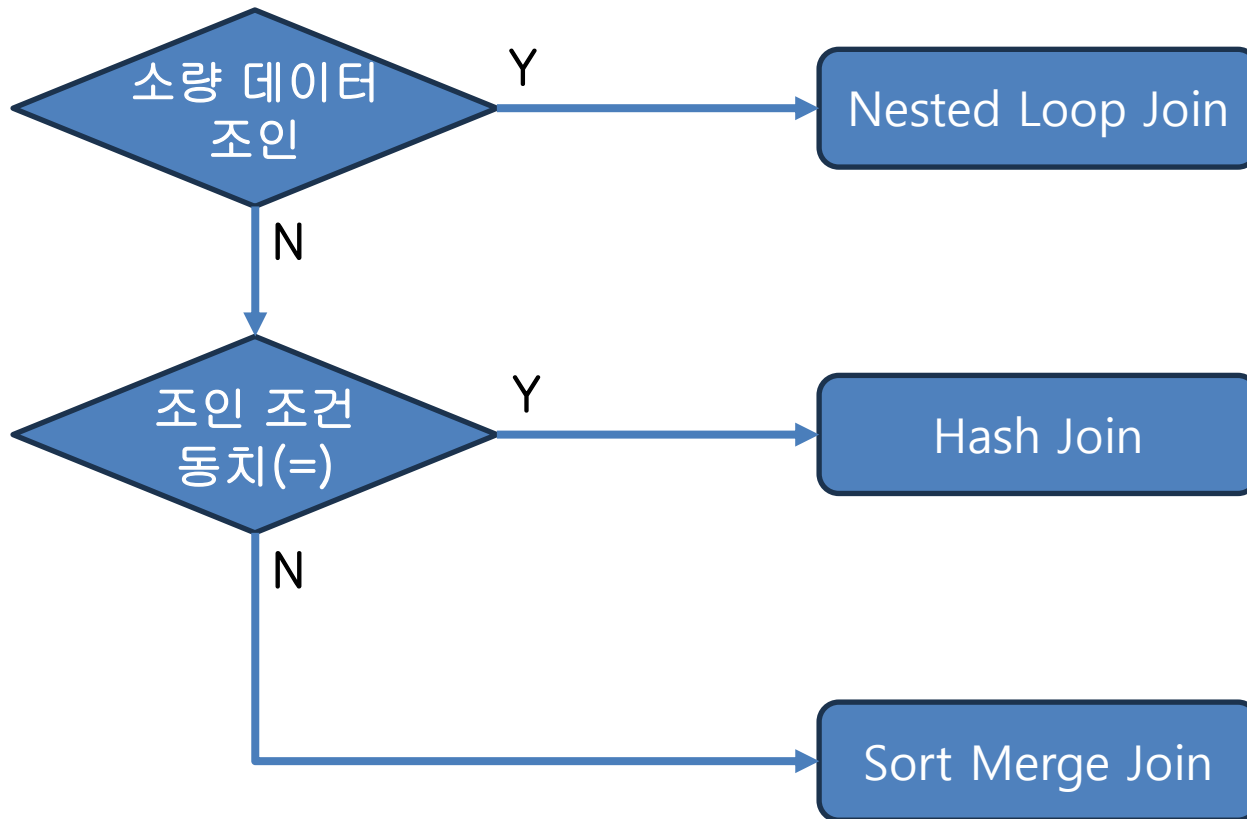
해시 조인은 해시테이블을 만드는 점 빼고는 NL 조인과 동일한 과정으로 조인

하지만 NL 조인보다 속도가 빠르는데 해시테이블이 PGA 영역에 생성되기 때문에 래치 획득 없이 PGA 에서 메모리상 데이터를 탐색해 조인하기 때문에 빠르다.

단, build input, probe input의 각 테이블을 읽을 때에는 DB 버퍼 캐시를 사용하여, 이 과정에서 발생하는 랜덤 액세스, 탐색비용은 발생 됨

12.4 조인 기법 선택 기준

데이터 기준



12.4 조인 기법 선택 기준

수행빈도 기준

수행빈도	조인 기법	기준
높음	NL 조인	- NL 조인과 Hash 조인의 성능이 동일 - NL 조인이 Hash 조인보다 빠른 경우
	Hash 조인	- Hash 조인이 NL 조인보다 빠른 경우
낮음	Hash 조인	- Query 수행시간이 오래 걸릴 경우 - 대량 데이터를 조인할 경우

- Hash 조인의 수행빈도가 높으면 메모리에 **해시테이블이 다수가 생성되어 CPU와 메모리 사용률이 증가**된다
- OLTP 환경에서는 NL 조인 사용이 유리
- OLAP, 배치, DW 환경에서는 Hash 조인 사용이 유리